Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Midterm Project

Author: Christian Lasek

Date: 10/11/2026

Abstract

This project asks whether a classifier can tell which AI model wrote a response. I gave the same 50 prompts to ChatGPT, Claude, and Gemini, collecting 150 responses. Then, I compared predictions based on the response alone, the prompt alone, and both together. I tested a simple text-classification baseline and the two required neural models: a CNN and an RNN (implemented as a bidirectional LSTM). The best response-only accuracy was 63.3%, achieved by both the baseline and CNN, compared with 33.3% for a three-class chance benchmark. Prompts alone did not help, and adding a prompt to a response did not improve performance very consistently. The findings suggest that the collected answers contain clues about their source, but the test set is too small for broad conclusions.

1. Goal and research questions

Different AI models can answer the same question in many different ways. They may differ in response length, word choice, formatting, or how much extra explanation they give. I wanted to find out whether a classifier could use these differences to predict the source model.

- **RQ1:** Can we identify the model using only its response? How do the CNN and RNN perform?
- **RQ2:** Does including the user's prompt improve the prediction?

The classifier chooses among ChatGPT, Claude, and Gemini. My goal is to test attribution within this dataset, not to claim that all AI-generated text can always be traced to its model.

2. Dataset

I used 50 prompts across five categories: explanations; summarization and rewriting; creative writing; coding and computer science; and mathematics and logic, and manually inputted each response from the three LLMs I used, ChatGPT, Claude, and Gemini into an Excel file, which was later converted into a csv. Each model received the same prompts, giving 50 responses per model and 150 total records. The file `data/llm_responses.csv` has five columns: `prompt_id`, `task_type`, `prompt`, `model_name`, and `response`.

I checked and made sure the data was valid in the code space with `src/inspect_data.py`. There are 150 rows, no missing fields, 50 different prompts, one answer from each model for every prompt, and no duplicate prompt/model pairs. I used shared prompts to make comparisons between the models fairer. Otherwise, the classifier might learn which topics were assigned to each model rather than learning from their answers.

During review, I noticed a few possible data-quality issues: some Gemini responses ask for Gmail permission or do not answer the question, some copied coding responses appear to have lost line breaks, and some Claude answers refer to earlier conversation context. These inconsistencies can make a model much more recognizable for reasons beyond its writing style.

3. Method

I ran each classifier with three kinds of input:

1. **Response only:** the model's answer.
2. **Prompt only:** the question or instruction given to the model.
3. **Prompt + response:** both pieces of text together.

I had to split the dataset by prompt ID, keeping all three responses to the same prompt in the same group. Ten prompts (30 responses in total) were held out as the final test set. The other 40 prompts (120 responses in total) were used to train the simple baseline. For the CNN and BiLSTM, 32 of those prompts were used for training and eight for validation. All three classifiers used the same test prompts that were held out. This allows the model to check performance on new prompts without letting the same question appear in both training and testing.

Models and training

- **Baseline (`src/baseline.py`):** TF-IDF turns the text into numeric features based on words and two-word phrases, while logistic regression uses those features to predict the model. This is simply beneficial for comparison purposes.
- **CNN (`src/cnn.py`):** This model learns short patterns with items such as nearby words or punctuation and uses them to classify the answer. It learned word representations, filters covering groups of three, four, or five tokens, and a final prediction layer.
- **RNN (`src/rnn.py`):** I used a bidirectional LSTM, which is an RNN that learns from what order the words are in within a particular sequence. It reads the text in both directions before predicting the source model.

The neural models use up to 256 tokens per example, batches of 16, and up to 15 epochs. They select a checkpoint using validation results and when improvement stalls, it stops. I report accuracy, which is the fraction of correct predictions, and macro-F1 which is a score that gives each model label equal weight. Since the three levels are represented equally, a 33% accuracy is the chance reference point.

4. Results

Input	Classifier	Accuracy	Macro-F1
Response only	TF-IDF baseline	63.3%	0.601
Response only	CNN	63.3%	0.611
Response only	BiLSTM	53.3%	0.504
Prompt only	TF-IDF baseline	33.3%	0.167
Prompt only	CNN	33.3%	0.240
Prompt only	BiLSTM	33.3%	0.285
Prompt + response	TF-IDF baseline	56.7%	0.528
Prompt + response	CNN	60.0%	0.593
Prompt + response	BiLSTM	63.3%	0.507

The source files are `results/baseline_metrics.csv`, `results/cnn_metrics.csv`, and `results/rnn_metrics.csv`.

RQ1: Can we identify which LLM generated a response?

For this dataset to a degree, yes. Every response-only classifier scored above the 33.3% chance reference. The baseline and CNN each identified 19 of 30 test responses correctly (63.3%). The CNN had a slightly

better macro-F1 than the baseline, but overall, they performed very similarly. The BiLSTM identified 16 of 30 correctly (53.3%). Therefore, the more complex models did not clearly beat the simple baseline.

The baseline correctly identified all ten Claude responses in the test set but only three of ten Gemini responses. This suggests that some model labels were harder for this baseline to distinguish. It does not tell us whether the CNN or BiLSTM made the same mistakes.

RQ2: Does the user's prompt help identify the LLM?

The prompt did not consistently help**.** Since each prompt was given to every model, the prompt alone does not identify which model answered. All three prompt-only experiments had 33.3% accuracy.

Adding the prompt to the response actually lowered accuracy for the baseline (63.3% to 56.7%) and CNN (63.3% to 60.0%). While BiLSTM accuracy rose from 53.3% to 63.3%, its macro-F1 didn't change much (0.504 to 0.507). These mixed results do not establish that adding prompts helps. The repeated prompt may add words that do not distinguish the models. As for long examples, placing it before the response may also push part of the response beyond the 256-token limit. Those are only some possible explanations to the results.

5. Limitations and lessons learned

The test set contains only 30 responses, so one changed prediction moves accuracy by about 3.3 percentage points. The results come from one train/test split and should not be treated as proof that one method is always better.

Differences in the way models display headings, length, follow-up questions, and formatting may be useful clues, but I did not test which clue mattered most. Refusals, tool-permission messages, interface text, and references to prior chats could also reveal how a response was collected rather than a consistent writing fingerprint. The findings apply to three model labels and these 50 prompts. They do not establish performance on newer model versions or entirely different tasks.

One of the most key lessons was to keep the data balanced and group the answers by prompt when splitting it. Also, I learned that a simple baseline matters. In this experiment, it matched the CNN's best response-only accuracy. With more time, I would collect more examples using documented model versions and fresh conversations, repeat the experiment across several grouped splits, and look more closely at mistakes by task category.

6. Conclusion

In this small project, responses contained enough differences for the best classifiers to predict their source more often than chance. The response-only CNN and baseline both reached 63.3% accuracy, while the CNN had the highest response-only macro-F1 at 0.611. Prompt text alone was not useful for identifying the model, and adding it did not reliably improve classification. The results answer RQ1 and RQ2 for the collected data without claiming that the same outcome would hold for every AI model or prompt. A larger dataset would be necessary for true conclusions on how a classifier can predict it's source.

Reproducibility

To reproduce the experiments from the repository root, install the required Python packages and run `python src/inspect_data.py`, `python src/baseline.py`, `python src/cnn.py`, and `python src/rnn.py`.